

Inigo Quilez :: articles :: sphere functions



Intro

If you are an engineer, you might have heard this before:

"Well, first let us consider a spherical cow..."

This is a humorous observation of the way engineers often operate. But the truth is that replacing precise problems that have only approximated, expensive or even unknown solution by approximated problems that do have an exact, cheap or known solution is sometimes a good strategy. And so, in the context of graphics, sometimes you can replace a cow, indeed, by a sphere (for example, if it's not too close to the camera, or if you are in an occlusion pass).

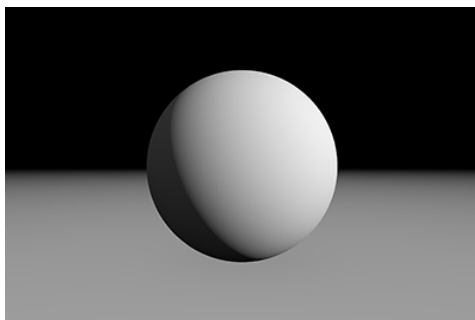
In fact, spheres are useful in too many ways to be ignored. I have written a few articles already on the subject of analytically computing different properties of spheres that are useful for rendering, which are scattered all over this website. Some others I have not documented yet, but this page can be seen as an index to those articles and techniques, and a summary of the resulting formulas. You can find them all also in this Shadertoy playlist: <https://www.shadertoy.com/playlist/sfcGWH>.

I have used these techniques both in demos and also in production of featured films. For example, I have used a bunch of spheres to shape its occlusion and directional shadows analytically rather than resorting to shadow maps. Also, if you are rendering massive terrains made of hundreds of millions of rocks, bushes and trees, you might want to organize things in bounding spheres and check analytically (without rasterization whatsoever), how many pixels does each bounding sphere take in screen space in to do level of detail decisions. I have used spherical density to do localized color grading, and for fog volumes.

Whether you are using spheres to approximate cows, or you are making a demo or game with spheres for artistic reasons, probably some of the formulas bellow will be of use to you.

Functions

In the code below, **pi** means 3.1415927... and **eps** is a small positive number close to zero.



Sphere Intersection

<http://iquilezles.org/blog/?p=2411>

```
float sphIntersect( vec3 ro, vec3 rd, vec4 sph )
{
    vec3 oc = ro - sph.xyz;
    float b = dot( oc, rd );
    float c = dot( oc, oc ) - sph.w*sph.w;
    float h = b*b - c;
    if( h<0.0 ) return -1.0;
    h = sqrt( h );
    return -b - h;
}

vec3 sphNormal( vec3 pos, vec4 sph )
{
    return (pos - sph.xyz)/sph.w;
}
```



Sphere Density

[Maths / Article](#)

[Example / Code](#)

```
float sphDensity( vec3 ro, vec3 rd, vec4 sph, float dbuffer )
{
    float ndbuffer = dbuffer/sph.w;
    vec3 rc = (ro - sph.xyz)/sph.w;

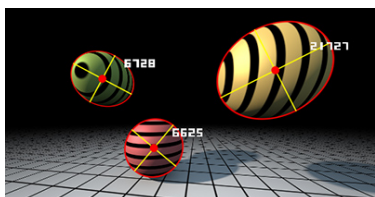
    float b = dot(rd,rc);
    float c = dot(rc,rc) - 1.0;
    float h = b*b - c;
    if( h<0.0 ) return 0.0;
    h = sqrt( h );
    float t1 = -b - h;
    float t2 = -b + h;
```

```

if( t2<0.0 || t1>ndbuffer ) return 0.0;
t1 = max( t1, 0.0 );
t2 = min( t2, ndbuffer );

float i1 = -(c*t1 + b*t1*t1 + t1*t1*t1/3.0);
float i2 = -(c*t2 + b*t2*t2 + t2*t2*t2/3.0);
return (i2-i1)*(3.0/4.0);
}

```



Sphere Projection

[Maths / Article](#)

[Example / Code](#)

```

float projectSphere( vec4 sph, mat4x4 cam, float fle )
{
    vec3 o = (cam*vec4(sph.xyz,1.0)).xyz;
    float r2 = sph.w*sph.w;
    float z2 = o.z*o.z;
    float l2 = dot(o,o);
    float k1 = l2-r2;
    float k2 = r2-z2;
    return -pi*fle*fle*r2*sqrt(abs(k1/k2))/k2;
}

```



Sphere Visibility

[Maths / Article](#)

[Example / Code](#)

```

int sphereVisibility( vec4 sA, vec4 sB, vec3 c )
{
    vec3 ac = sA.xyz - c;
    vec3 bc = sB.xyz - c;
    float ia = 1.0/length(ac);
    float ib = 1.0/length(bc);
    float k0 = dot(ac,bc)*ia*ib;
    float k1 = sA.w*ia;
    float k2 = sB.w*ib;
    float m1 = k0*k0 + k1*k1 + k2*k2;
    float m2 = 2.0*k0*k1*k2;

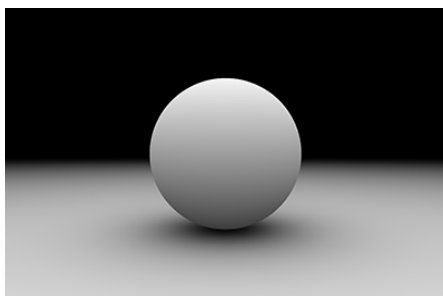
    if( m1 + m2 - 1.0 < 0.0 ) return 1;
    else if( m1 - m2 - 1.0 < 0.0 ) return 2;
}

```

```

        return 3;
    }

```



Sphere Ambient Occlusion

[Maths / Article](#)

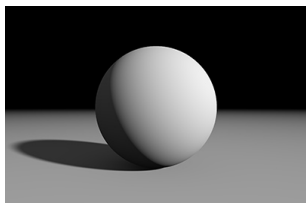
[Example / Code](#)

```

float sphOcclusion( vec3 pos, vec3 nor, vec4 sph )
{
    vec3 di = sph.xyz - pos;
    float l = length(di);
    float nl = dot(nor, di/l);
    float h = 1/sph.w;
    float h2 = h*h;
    float k2 = 1.0 - h2*nl*nl;

    float res = max(0.0, nl)/h2;
    if( k2 > 0.0 ) // approx. for penetration
    {
        res = clamp(0.5*(nl*h+1.0)/h2, 0.0, 1.0);
        res = sqrt( res*res*res );
    }
    return res;
}

```



Sphere Soft Shadow

[Maths / Article](#)

[Example / Code](#)

```

float sphSoftShadow( vec3 ro, vec3 rd, vec4 sph, float k )
{
    vec3 oc = ro - sph.xyz;
    float b = dot( oc, rd );
    float c = dot( oc, oc ) - sph.w*sph.w;
    float h = b*b - c;
    return (b>0.0)?step(-eps, c):smoothstep(0.0, 1.0, h*k/b);
}

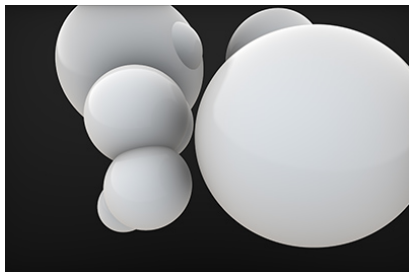
float sphSoftShadow( vec3 ro, vec3 rd, vec4 sph, float k )
{
    vec3 oc = ro - sph.xyz;
    float r = sph.w*sph.w;
    float b = dot( oc, rd );
    float c = dot( oc, oc ) - r;
    float h = b*b - c;

```

```

float d = -sph.w + sqrt( max(0.0,r-h));
float t = -b          - sqrt( max(0.0,h) );
return (t<0.0)?1.0:smoothstep(0.0,1.0,k*d/t);
}

```



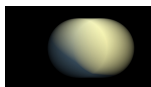
Sphere Distance

Example / Code

```

vec2 sphDistances( vec3 ro, vec3 rd, vec4 sph )
{
    vec3 oc = ro - sph.xyz;
    float b = dot( oc, rd );
    float c = dot( oc, oc ) - sph.w*sph.w;
    float h = b*b - c;
    float d = sqrt( max(0.0,sph.w*sph.w-h)) - sph.w;
    return vec2( d, -b-sqrt(max(h,0.0)) );
}

```



Sphere Motion

Example / Code

```

vec2 sphMotion( vec3 ro, vec3 rd, vec4 sph, vec3 ve, out vec3 nor )
{
    float t = -1.0;
    float s = 0.0;
    nor = vec3(0.0);

    vec3 rc = ro - sph.xyz;
    float A = dot(rc,rd);
    float B = dot(rc,rc) - sph.w*sph.w;
    float C = dot(ve,ve);
    float D = dot(rc,ve);
    float E = dot(rd,ve);
    float aab = A*A - B;
    float eec = E*E - C;
    float aed = A*E - D;
    float k = aed*aed - eec*aab;

    if( k>0.0 )
    {
        k = sqrt(k);
        float hb = (aed - k)/eec;
        float ha = (aed + k)/eec;
        float ta = max( 0.0, ha );
        float tb = min( 1.0, hb );
    }
}

```

```
if( ta < tb )
{
    ta = 0.5*(ta+tb);
    float k1 = A - E*ta;
    float k2 = B + C*ta*ta - 2.0*D*ta;
    t = -k1 - sqrt( k1*k1 - k2 );
    s = 2.0*(tb - ta);

    vec3 pa = ro + rd*t;
    vec3 pb = sph.xyz + ta*ve;
    nor = normalize( pa - pb );
}

return vec2(t,s);
}
```

inigo quilez - learning computer graphics since 1994